



Contents lists available at ScienceDirect

Theoretical Computer Science

journal homepage: www.elsevier.com/locate/tcs

Generating Java code pairing with ChatGPT ☆☆☆

Zelong Zhao, Nan Zhang, Bin Yu, Zhenhua Duan*

Institute of Computing Theory and Technology, and ISN Laboratory, Xidian University, No. 266 Xinglong Section of Xi Feng Rd, Xi'an, 710126, Shaanxi, China

ARTICLE INFO

Keywords:

Code generation
Large language models
ChatGPT
Prompt engineering
Iterative prompting

ABSTRACT

The Large Language Models (LLMs) like ChatGPT 3.5 have created a new era of automatic code generation. However, the existing research primarily focuses on generating simple code based on datasets (such as HumanEval, etc.). Most of approaches pay less attention to complex and practical code generation. Therefore, in this paper, we propose a new approach called “Xd-CodeGen” which can be used to generate large scale Java code. This approach is composed of four phases: requirement analysis, modeling, code generation, and code verification. In the requirement analysis phase, ChatGPT 3.5 is utilized to decompose and restate user requirements. To do so, a knowledge graph is developed to describe entities and their relationship in detail. Further, Propositional Projection Temporal Logic (PPTL) formulas are employed to define the properties of requirements. In the modeling phase, we use knowledge graphs to enhance prompts and generate UML class and activity diagrams for each sub-requirement using ChatGPT 3.5. In the code generation phase, based on established UML models, we make use of prompt engineering and knowledge graph to generate Java code. In the code verification phase, a runtime verification at code level approach is employed to verify generated Java code. Finally, we apply the proposed approach to develop a practical Java web project.

1. Introduction

As software becomes increasingly complex and larger, it is difficult to ensure reliability of software. The Automatic Code Generation (ACG) technology has significantly reduced the workload of manual coding, improved development efficiency, and decreased coding errors. However, traditional ACG suffers from requirements analysis, modeling, coding and testing processes.

Recently, Large Language Models (LLMs) based on the transformer [1] architecture, such as ChatGPT 3.5 [2], GPT-4 [3], Llama 2 [4], and others, have created a new era of AI and intelligent information technologies. These LLMs are capable of understanding human intentions and generating human-like responses, demonstrating impressive performance in a variety of tasks, such as text summarization [5], machine translation [6], language understanding [7], program coding and more.

In light of the excellent capabilities of LLMs in text generating and understanding, software engineers have tried to make use of the LLMs for various Software Engineering (SE) tasks [8–14]. With the advancement of LLMs technologies, significant breakthroughs have been made in ACG, an important SE subfield. Currently, requirements can be described in natural language, and LLMs can be utilized to understand these requirements, generating corresponding code snippets. This can significantly reduce the manual coding

☆ This article belongs to Section A: Algorithms, automata, complexity and games, Edited by Paul Spirakis.

☆☆ This research is supported by the National Natural Science Foundation of China with Grant No. 62192734, 62172322, and 62272359.

* Corresponding author.

E-mail addresses: zhaozl@stu.xidian.edu.cn (Z. Zhao), nanzhang@xidian.edu.cn (N. Zhang), byu@xidian.edu.cn (B. Yu), zhhdian@mail.xidian.edu.cn (Z. Duan).

<https://doi.org/10.1016/j.tcs.2024.114879>

Received 2 April 2024; Received in revised form 22 July 2024; Accepted 12 September 2024

Available online 17 September 2024

0304-3975/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

workload. With the LLMs, researchers in ACG area mainly focus on two code generation paradigms: “pre-train, fine-tune” and “pre-train, prompt, predict” [15].

In the “pre-train, fine-tune” paradigm, researchers utilize code from open-source platforms like GitHub to fine-tune models to meet specific code generation tasks [9][16–18]. While this approach achieves good performance with less data, it requires retraining the model, consuming significant computational resources, and may result in performance degradation in certain scenarios. In contrast, the “pre-train, prompt, predict” paradigm improves code generation performance by optimizing prompt design, avoiding model adjustments. This is due to the high dependency of language models on prompt quality and better utilization of contextual information in text generation.

In terms of “pre-train, prompt, predict” paradigm, current academic research primarily revolves around benchmark datasets such as HumanEval [9] and MBPP [19]. Researchers employ Prompt Engineering (PE) techniques to enhance the performance of existing models on benchmark datasets, which mainly consist of programming competition problems not found in internet resources. Due to the simplicity of the task descriptions in these studies, the focus is primarily on the coding phase, neglecting other stages in the software lifecycle. The methods proposed in these studies are challenging for coding complex software systems. For complex programming problems, Ahmad et al. [10] and Cmara et al. [20] explore the capabilities of LLMs in software modeling but do not delve into subsequent coding exploration. Dong et al. [21] introduce a self-collaborative architecture where different instances of ChatGPT are responsible for the design, coding, and testing phases of the software development processes, collaborating to complete the coding task. However, the emphasis is on collaboration between different ChatGPT instances, and the experiments in the paper are still centered around benchmark datasets. In [22], Unified Modeling Language (UML) diagrams and constraint languages are used to generate high-level language code, with LLMs playing the role of code generators. In [23], the authors employ three developers with varying levels of experience to use ChatGPT for completing a Q&A forum, providing an experiential report rather than a systematic approach. These studies explore various aspects of integrating LLMs into the software development processes, but they do not develop a systematic method for generating complete system code based on user requirements of complex programming tasks.

To address the above problems, we propose an approach named “Xd-CodeGen” to solve complex code generation problems in the practical applications. This approach employs ChatGPT 3.5 and interacts with it through iterative prompting, dividing the entire code generation process into four phases: requirement analysis, modeling, code generation, and code verification. In the requirement analysis phase, ChatGPT 3.5 is utilized to decompose and restate user requirements. To do so, a knowledge graph is developed to describe entities and their relationship in detail. Further, Propositional Projection Temporal Logic (PPTL) formulas are employed to define the properties of requirements. In the modeling phase, we use knowledge graphs to enhance prompts and generate UML class and activity diagrams for each sub-requirement using ChatGPT 3.5. In the code generation phase, based on established UML models, we make use of prompt engineering and knowledge graph to generate Java code. In the code verification phase, a runtime verification at code level approach is employed to verify generated Java code. Finally, we apply the proposed approach to develop a practical Java web project.

The main contributions of this paper are as follows:

(1) A novel Java code generation approach called “Xd-CodeGen” is proposed. It divides the entire code generation process into four phases: requirement analysis, modeling, code generation, and code verification. The four phases are completed pairing with ChatGPT 3.5. This approach is suitable for generating large scale Java code.

(2) The approach utilizes LLMs and knowledge graph for software requirement analysis. In this way, it enables ChatGPT to well understand requirements. Further, knowledge graph allows us to enhance prompt accuracy during the modeling and code generation phases.

(3) This approach uses UML models, class diagram and activity diagram to modeling system to be coded. During the code generation phase, the established UML models are used as prompts to generate code. At the same time, UML models can serve as software documentation, improving the visibility of the software development process.

(4) The proposed approach employs the runtime verification at code level approach to verify the generated code to ensure its reliability. A Java web project was developed using the proposed approach, demonstrating the effectiveness of the approach.

This paper is organized as follows: the next section introduces the fundamental knowledge of software development, ChatGPT, prompt engineering, PPTL and runtime verification at code level. Section 3 introduces the proposed “Xd-CodeGen” approach in detail. Section 4 describes the application of a practical Java web project developed using “Xd-CodeGen”. Finally, conclusions are drawn in Section 5.

2. Preliminaries

This section will introduce the fundamental process of software development and provide relevant knowledge about ChatGPT, Prompt Engineering, Propositional Projection Temporal Logic (PPTL) and Runtime Verification at Code Level.

2.1. Software development process

The software development process typically includes requirement analysis, system design, detailed design, coding, testing, verification and operational maintenance [24].

Requirement analysis is intended to convert users’ intentions and ideas into technical specifications required by software developers. System design clarifies the construction of the system based on requirements analysis, including data structure design, system

architecture design, interface design, and process design. Detailed design refers to the data structure design and algorithm design among various modules of the software system.

The purpose of coding is to convert the system design into source code using the selected programming language. Coding should follow three basic principles: appropriate language selection, consistent coding style, and efficient code implementation. Software testing is the process of executing a program to find errors, and serves as the review of the requirement analysis, design, and coding stages. Program verification aims to improve the reliability of computer programs by using mathematical and logical means. It involves the use of formal methods to prove whether the behavior of a program, under specific specifications, conforms to the expected requirements. In practical applications, program verification is often combined with testing and other software development methods to ensure the quality and reliability of the software.

2.2. ChatGPT

ChatGPT is a novel language model developed by OpenAI designed for interactive conversations. It is a type of LLMs based on the transformer architecture. The fundamental principles involve data preprocessing, pre-training the transformer model, and fine-tuning using Reinforcement Learning from Human Feedback (RLHF) [25]. ChatGPT exhibits unparalleled natural language processing capabilities, and generates not only fluent and coherent text but also comprehending and manipulating text in various ways. Additionally, it demonstrates high proficiency in programming tasks, capable of handling diverse programming challenges. It can understand the grammar and semantics of natural language, extract crucial information, and program accordingly to fulfill specific requirements.

However, ChatGPT may occasionally generate syntactically invalid or semantically incorrect code, especially for longer or more complex programs, leading to discrepancies with the expected functionality or style. Despite this, through the error messages from compilers or terminals and feedback from humans, it can iteratively receive prompt guidance, improving its generated code with each iteration, gradually approaching to the final correct code.

2.3. Prompt engineering

PE is an emerging discipline that has evolved with the advancement of LLMs [26]. Its purpose is to design and optimize prompts to guide LLMs in generating outputs consistent with various application scenarios and research topics. PE involves selecting suitable vocabulary, grammar, context, and themes, and employs multiple techniques and strategies to influence the generative behavior and outcomes of models. By optimizing PE, we can well understand the capabilities and limitations of LLMs, and make these LLMs models more accurate, controllable, and adaptable to different tasks and application scenarios.

A prompt can include any combination of the following components: 1) instruction: the specific task you want the model to execute; 2) context: external information or additional context to help the model respond well; 3) input data: the input or question of interest; 4) output indicators: the type or format of the output. Not all components need to be included in a prompt, and its format depends on the task. There are two key principles in writing prompts: principle 1: formulating clear and precise instructions in order to minimize the possibility of irrelevant or incorrect responses; principle 2: allowing the model ample processing time, since requiring a response in a short time with limited vocabulary can lead to inaccurate results. Designing prompts also involves using various techniques, such as decomposing complex tasks with multiple subtasks into simpler components; using clear separators like “###” to distinguish instructions and context, and considering the length of the prompt, as there are limitations to prompt length. Simultaneously, there are some advanced PE techniques, including zero-shot prompting [27], few-shot prompting [28], Chain of Thought (CoT) prompting [29,30], zero-shot CoT [27], and self-consistency [31].

2.4. Propositional projection temporal logic

Our underlying logic is Propositional Projection Temporal Logic (PPTL) [32,33].

Syntax Let $Prop$ be a set of atomic propositions and $p \in Prop$. The syntax of PPTL is inductively defined as follows.

$$P ::= p \mid \bigcirc P \mid \neg P \mid P \vee Q \mid (P_1, \dots, P_m) \text{ } prj \text{ } Q \mid P^+$$

where $P_1, \dots, P_m, P$ and Q are well-formed PPTL formulas. Here, $\bigcirc$ (next), prj (projection) and $+$ (chop-plus) are primitive temporal operators. A formula is called a *state formula* if it contains no temporal operators, otherwise it is called a *temporal formula*.

Semantics Let $B = \{\top, \perp\}$ be the set of boolean values and N the set of non-negative integers. Let ω denote infinity. We consider the set $N_\omega = N \cup \{\omega\}$ and extend the comparison operators, $=, <, \leq$, to N_ω by considering $\omega = \omega$, and for all $i \in N$, $i < \omega$. Furthermore, we define $\leq$ as $\leq - \{(\omega, \omega)\}$.

PPTL formulas are interpreted over intervals. An interval σ is a finite or infinite sequence of states, denoted by $\sigma = \langle s_0, s_1, \dots \rangle$. A state s_i is a mapping from $Prop$ to B . An interpretation $I = (\sigma, k, j)$ is a subinterval $\langle s_k, \dots, s_j \rangle$ of σ with the current state being s_k . The length, $|\sigma|$, of σ is ω if σ is infinite, and the number of states minus 1 if σ is finite.

An auxiliary operator $\downarrow$ is defined as $\sigma \downarrow (r_1, \dots, r_m) = \langle s_{t_1}, s_{t_2}, \dots, s_{t_n} \rangle$, where $r_1 \leq \dots \leq r_m \leq |\sigma|$ and $r_i \in N$ ($i = 1, \dots, m$), and $t_1, \dots, t_n$ are obtained from $r_1, \dots, r_m$ by deleting all duplicates. That is, $t_1, \dots, t_n$ is the longest strictly increasing subsequence of $r_1, \dots, r_m$. For example,

$$\sigma \downarrow (2, 2, 3, 5, 5, 7) = \langle s_2, s_3, s_5, s_7 \rangle$$

The semantics of PPTL formulas is inductively defined as a satisfaction relation below.

- (1) $I \models p$ iff $s_k[p] = \top$.
- (2) $I \models \bigcirc P$ iff $k < j$ and $(\sigma, k+1, j) \models P$.
- (3) $I \models \neg P$ iff $I \not\models P$.
- (4) $I \models P \vee Q$ iff $I \models P$ or $I \models Q$.
- (5) $I \models (P_1, \dots, P_m) \text{ prj } Q$ iff there exist m integers $k = r_0 \leq r_1 \leq \dots \leq r_m \leq j$ such that $(\sigma, r_{l-1}, r_l) \models P_l$ for all $1 \leq l \leq m$, and one of the following two cases holds:
 - if $r_m = j$, there exists r_h such that $0 \leq h \leq m$ and $\sigma \downarrow (r_0, \dots, r_h) \models Q$;
 - if $r_m < j$, then $\sigma \downarrow (r_0, \dots, r_m) \cdot \sigma_{(r_m+1..j)} \models Q$.
- (6) $I \models P^+$ iff there exist m integers $k = r_0 \leq r_1 \leq \dots \leq r_m = j$ ($m \in \mathbb{N}$ and $m \geq 1$) such that $(\sigma, r_{l-1}, r_l) \models P$ for all $1 \leq l \leq m$.

The interpretation of $(P_1, \dots, P_m) \text{ prj } Q$ is that we need two sequences of clocks (states) running on different time scales: one is a local state sequence, over which $P_1; \dots; P_m$ are interpreted, the other is a global state sequence over which Q is interpreted. Formula Q is interpreted in a parallel manner with the sequence of formulas $P_1; \dots; P_m$. The interpretation proceeds as follows: First, Q and P_1 start at the first global state and P_1 is interpreted over a sequence of local states until its termination. Then (the remaining part of) Q and P_2 are executed at the second global state. Subsequently, P_2 is continuously interpreted over a sequence of local states until its termination, and so on. Although Q and P_1 start at the same time, $P_1; \dots; P_m$ and Q may terminate at different time points. If Q terminates before some P_{h+1} , then, subsequently, $P_{h+1}; \dots; P_m$ are interpreted sequentially. If $P_1; \dots; P_m$ are finished before Q , then the execution of Q is continued until its termination. Note that the projection construct can be interpreted over an infinite interval. Projection can be thought of as a special parallel computation which is executed on different time scales.

The operators *true*, *false*, $\wedge$, $\rightarrow$ and $\leftrightarrow$ are defined as usual. In particular, $\text{true} \stackrel{\text{def}}{=} P \vee \neg P$ and $\text{false} \stackrel{\text{def}}{=} P \wedge \neg P$ for any formula P . The derived formulas are given as follows, where $n \in \mathbb{N}_0$ (set of natural numbers).

A1	<i>more</i>	$\stackrel{\text{def}}{=} \bigcirc \text{true}$	A2	ϵ	$\stackrel{\text{def}}{=} \neg \bigcirc \text{true}$
A3	$\bigcirc^0 P$	$\stackrel{\text{def}}{=} P$	A4	$\bigcirc^{n+1} P$	$\stackrel{\text{def}}{=} \bigcirc(\bigcirc^n P)$
A5	$P_1; P_2$	$\stackrel{\text{def}}{=} (P_1, P_2) \text{ prj } \epsilon$	A6	$\Diamond P$	$\stackrel{\text{def}}{=} \text{true}; P$
A7	$\Box P$	$\stackrel{\text{def}}{=} \neg \Diamond \neg P$	A8	$\odot P$	$\stackrel{\text{def}}{=} \epsilon \vee \bigcirc P$
A9	P^0	$\stackrel{\text{def}}{=} \epsilon$	A10	P^{n+1}	$\stackrel{\text{def}}{=} P^n; P$
A11	<i>len</i> (n)	$\stackrel{\text{def}}{=} \bigcirc^n \epsilon$	A12	<i>skip</i>	$\stackrel{\text{def}}{=} \text{len}(1)$
A13	<i>fin</i> (p)	$\stackrel{\text{def}}{=} \Box(\epsilon \rightarrow p)$	A14	<i>inf</i>	$\stackrel{\text{def}}{=} \neg \Diamond \epsilon$
A15	<i>keep</i> (p)	$\stackrel{\text{def}}{=} \Box(\text{more} \rightarrow p)$	A16	<i>halt</i> (p)	$\stackrel{\text{def}}{=} \Box(\epsilon \leftrightarrow p)$

2.5. Runtime verification at code level

Runtime verification at code level [34] is a code verification method based on Modeling, Simulation, Verification Language (MSVL) [35,36]. The specific implementation steps of this method can be summarized as follows: firstly, we formalize real-life software and hardware systems using an MSVL program M . Then, we describe the property to be verified in the system using a PPTL formula P . Next, to determine whether the system satisfies the selected property for verification, we need to prove satisfiability of the formula $M \rightarrow P$. If the formula $M \rightarrow P$ is satisfiable, it indicates that the system satisfies the property; otherwise, it means that the system violates the property P . Since $\neg(M \rightarrow P) \equiv M \wedge \neg P$, we can indirectly determine whether the system satisfies the property by checking the satisfiability of $M \wedge \neg P$. If $M \wedge \neg P$ is unsatisfiable, it indicates that the system satisfies the property; otherwise, it does not.

Both the MSVL program used to describe system models and the PPTL formulas used to describe certain properties in the system can be transformed into their respective canonical forms. These canonical forms, in terms of their structure, exhibit similarities as both consist of two parts: one part represents what holds true in the current state, while the other part represents what holds true in the next state. Additionally, MSVL and PPTL both belong to the Projection Temporal Logic (PTL) framework, which means that they can be converted into PTL formulas. Consequently, they can be interpreted and executed within the same MSV platform [35] to perform simulation, modeling, and property verification. For each interval $\sigma \models M \wedge \neg P$, it allows us to determine a counterexample path in which the system violates the property. The MSV platform returns a Normal Form Graph (NFG), ultimately allowing us to determine whether the system satisfies the property under verification.

3. Xd-CodeGen

This section introduces our proposed approach: Xd-CodeGen. We interact with ChatGPT using an iterative prompting method and apply the principles of MDD to illustrate the approach as four phases: requirement analysis, modeling, code generation, and code verification. Fig. 1 shows an outline of the approach.

In Fig. 1, the four colored rectangular blocks represent the tasks to be completed in each phase. In the requirement analysis phase, we first send the user requirements R to ChatGPT and, by means of prompts, ask it to decompose R into sub-requirements

Algorithm 1 ChatGPTGen.**Input:** string prompt P **Output:** C_d

```

1: function CHATGPTGEN( $P$ )
2:   ChatGPT executes prompt  $P$ .
3:   assign the returned result from ChatGPT to  $C_d$ .
4:   return  $C_d$ 
5: end function

```

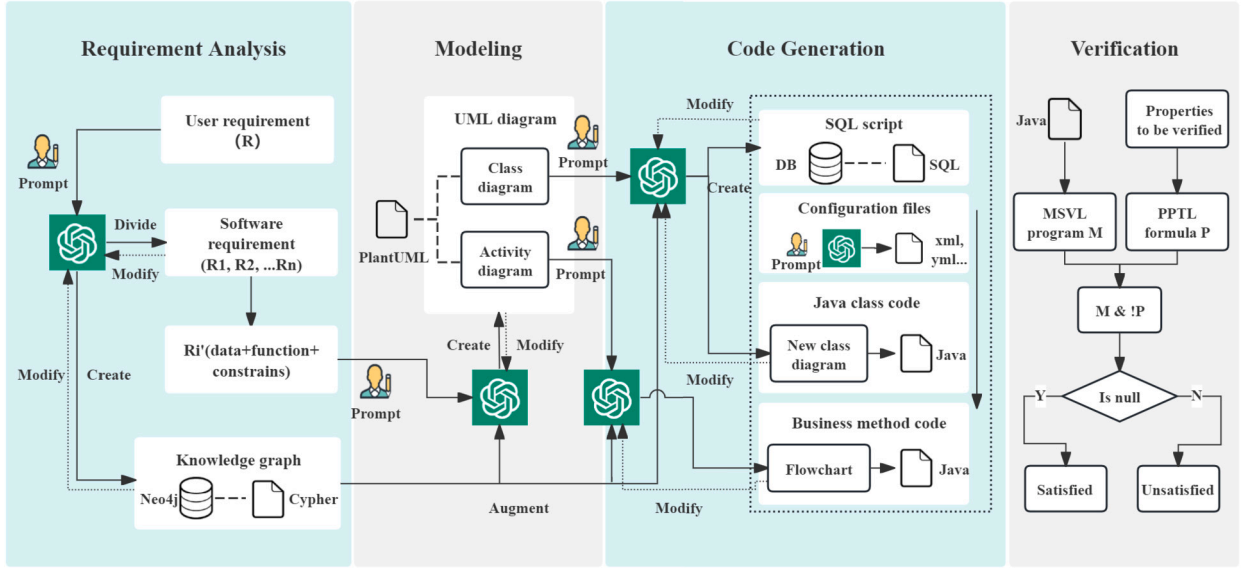


Fig. 1. Outline of Xd-CodeGen.

$R_0, R_1, \dots, R_n$; simultaneously, we prompt it to create a business knowledge graph K , with Cypher being the expression form of K , and Neo4j representing the storage approach of K . Next, we re-describe the sub-requirements $R_i (0 \leq i \leq n)$ as R'_i which reconstructs the contents in R_i more structured according to data types, functionalities, and constraints. The dashed arrows indicate an iterative modification process, that is, when we are unsatisfactory with the results generated by ChatGPT, we ask it to regenerate or modify the results. In the modeling phase, we send the sub-requirements R'_i to ChatGPT, prompting it to perform UML modeling, and resulting class diagrams and activity diagrams. Dotted lines without arrows pointing to PlantUML indicate that the expression form of UML diagrams is PlantUML script. Meanwhile, we use K to provide richer information to enhance ChatGPT's modeling capabilities. In the code generation phase, we take the UML diagrams and knowledge graph K as inputs and prompt ChatGPT to generate code. First, ChatGPT produces SQL scripts and we execute them in a relational database; secondly, ChatGPT creates configuration files required by the project, then modifies the original class diagrams to obtain new class diagrams compatible with the development framework, and generates Java class code based on the new class diagrams; thirdly, ChatGPT creates a flowchart for business functions based on the activity diagrams, and generates business logic code according to the flowchart. Finally, in the code verification phase, we employ the runtime verification at code level approach to verify the generated code.

3.1. Requirement analysis

User requirements are typically obtained by interaction between software developers and users. We refer to the initial user requirements as R . Our goal is to prompt ChatGPT to generate Java code that satisfies R . In practice, humans are responsible for writing the prompts, and ChatGPT is responsible for writing the code. According to reports in literature [18], when faced with complex requirements and software system designs, current LLMs are unable to provide a comprehensive solution. Furthermore, by our experience, when ChatGPT acts as a software engineer, and we input R as a prompt and request it to generate complete code satisfying R , as a result, natural language descriptions of solution steps, or snippets of code are obtained. However, these outcomes are sometimes unrelated to the final implementation. In other words, it is not possible for ChatGPT to generate code that satisfies requirement R in a single interaction.

Therefore, we adopt a divide-and-conquer method, decomposing requirement R based on data-centric principles. This involves breaking down R into various data blocks according to the system's data management needs. Each block is responsible for managing data with a specific type, including data input, data storage, data modification, and data querying, while adhering to principles such as high cohesion, low coupling, single responsibility, and traceability. We employ ChatGPT to accomplish this step with input R and decomposition principles (P_d). Accordingly, the decomposed requirements $R_0, R_1, \dots, R_n$ are obtained. Here, $R = \bigcup_i R_i, R_i (0 \leq i \leq n)$

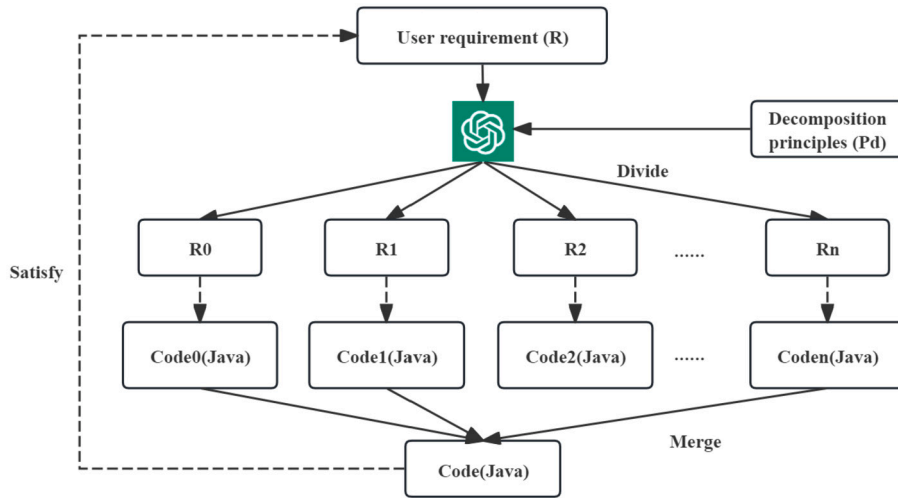


Fig. 2. Requirement decomposition.

represents sub-requirements as illustrated in Fig. 2. This allows us to decompose R into relatively independent blocks. We then guide ChatGPT to gradually complete these blocks. Finally, the completed blocks are unified to satisfy R . However, based on our experience, even with division and conquer, it is still not feasible for ChatGPT to transform R_i into executable code in one step.

Due to LLMs memorizing world knowledge parametrically and failing to refer to concrete knowledge sources, during the requirement analysis phase, we establish a knowledge graph K to describe the entity relationships within R . In the modeling and code generation phases, K is utilized to provide domain knowledge, enhancing ChatGPT's understanding of the business. A knowledge graph is a technique for storing and managing complex entities and their interrelationships. The fundamental elements in a knowledge graph are entities (nodes) and the relationships (edges) between them. Additionally, it can include rich semantic information such as attributes, classifications, and hierarchies. We employ ChatGPT to assist in building knowledge graph K . Since ChatGPT only accepts text input, we need a textual representation of the knowledge graph that ChatGPT can comprehend. Ultimately, we use the Neo4j graph database to construct the knowledge graph. Neo4j is a popular graph database management system suitable for building knowledge graphs, and Cypher is the query language for Neo4j. The established knowledge graph is represented in the form of Cypher scripts. When guiding ChatGPT's modeling and code generation, we first input the Cypher script to provide contextual information, and then proceed with entering task instructions. We utilize ChatGPT's in-context learning ability to impart domain knowledge.

The process of constructing a knowledge graph is illustrated by Algorithm 2. Here, ChatGPTGen is defined as shown in Algorithm 1, representing the process of ChatGPT responding to prompts. Firstly, R is sent to ChatGPT in batches as prior knowledge. Next, the role instruction I_r is used, with its content "Starting now, you will take on the role of a requirement analyst, and you are very familiar with Neo4j databases as well as Cypher query scripts." ChatGPT is then prompted to extract information from R and is requested to return the information in the order of (E, RI, A) . E , RI , and A represent sets of entities, relationships, and attributes, respectively. Items within the entity set E , denoted as E_i , a tuple consisting of the entity name and an optional explanation. Each item in the relationship set RI is a tuple of (E_i, RI_i, E_j) , where E_i and E_j represent different entities, and RI_i represents the relationship between E_i and E_j . Each item in the attribute set A is a tuple consisting of the entity name and a list of attributes, representing all attributes related to each entity. The triple (E, RI, A) extracted by ChatGPT is also modified and supplemented by ChatGPT as well, and ChatGPT is iteratively prompted to generate Cypher scripts based on (E, RI, A) . The final result is the generated Cypher scripts. Subsequently, we execute the Cypher scripts in Neo4j. In the section 4, Fig. 9 displays a part of the knowledge graph we constructed.

As this paper focuses solely on the functionality correctness and executability of the code, we refine the requirement R_i using three sets: the dataset D_i , the function set F_i , and the constraint set C_i . Here, D_i represents all the user data to be recorded in R_i along with their explanations; F_i stands for all the functionalities to be implemented in R_i with explanations; C_i describes the constraints that the data in D_i and the functionalities in F_i need to satisfy. Simultaneously, we delete irrelevant redundant data in R_i and refer to the refined requirement as R'_i . This refinement serves two purposes. Firstly, it extracts the key requirements so that ChatGPT is able to more accurately and effectively to generate code within token limitations. Secondly, it allows us to choose a subset of R'_i in subsequent modeling and coding processes.

3.2. Modeling

We use UML to model a system to be developed. UML diagrams have a textual representation known as PlantUML scripts. In light of ChatGPT's ability to generate and understand PlantUML scripts, we make use of UML diagrams, class diagram and activity diagram, as a transforming bridge between requirements and codes. In this way, we can not only improve efficiency of ACG but also obtain the design documentation. This approach enables us to prompt ChatGPT to generate PlantUML scripts based on refined requirement R'_i and use third-party tool (planuml.jar) [37] to visualize the scripts. Subsequently, PlantUML scripts can be used to generate Java

Algorithm 2 Creating Knowledge Graph.

Input: user requirement R , role instruction I_r , error set E_s
Output: knowledge graph Cypher script C_d

```

1: function CREATEKG( $R, I_r, E_s$ )
2:   initialize  $n$  as the number of batches.
3:   for  $i = 1 \rightarrow n$  do
4:     CHATGPTGEN( $R_i$ )
5:   end for
6:   CHATGPTGEN( $I_r$ )
7:    $T \leftarrow$  a string prompt for extracting message.
8:    $C'_d \leftarrow$  CHATGPTGEN( $T$ )
9:   manually modify and supplement  $C'_d$ .
10:   $T' \leftarrow$  a string prompt for generating Cypher script.
11:   $P_0 \leftarrow C'_d + T'$ 
12:   $C_d \leftarrow$  ITERATIVEPROMPT( $P_0, E_s$ )
13:  return  $C_d$ 
14: end function

```

R_s : I would like you to help me create a UML class diagram. You need to generate the Plantuml script corresponding to the UML class diagram.

R_q : What information do you need from me to complete the R_s ?

Fig. 3. Example of R_s and R_q .

code. Using UML modeling techniques can reduce the common ambiguities in natural language descriptions so as to ChatGPT can correctly understand prompts. Additionally, the graphical representation of UML diagrams facilitates error review and corrections.

Class diagrams depict the static structure of classes and the relationships between them in the system. Each class is represented by a rectangle, containing the class name, attributes, and methods. Activity diagrams describe the dynamic flow of business processes or system operations. We have chosen class diagram and activity diagram because R'_i contains Di and Fi which are closely related to class diagrams and activity diagrams. Additionally, in the coding phase, UML diagrams will be used in script form as prompts. Considering the token limitations of ChatGPT, we are unable to use too many types of UML diagrams in the coding phase.

Algorithm 3 Generating UML Diagrams.

Input: R_s, R_q , role instruction I_r , a string list L , error set E_s
Output: PlantUML script C_d

```

1: function GENERATEUMLDIAGRAMS( $R_s, R_q, I_r, L, E_s$ )
2:   $P_{sq} \leftarrow R_s + R_q$ 
3:  for  $i = 1 \rightarrow 3$  do
4:    CHATGPTGEN( $I_r$ )
5:     $L_i \leftarrow$  CHATGPTGEN( $P_{sq}$ )
6:     $L \leftarrow L + L_i$ 
7:  end for
8:  initialize  $P_0$  based on  $R'_i, K$  and  $L$ 
9:  initialize  $E_s$ 
10: CHATGPTGEN( $K$ )
11: CHATGPTGEN( $I_r$ )
12:  $C_d =$  ITERATIVEPROMPT( $P_0, E_s$ )
13: return  $C_d$ 
14: end function

```

Algorithm 3 describes how to generate class diagrams and activity diagrams, where I_r denotes role instructions specifying the roles played by ChatGPT. R_s represents a prompt to generate PlantUML scripts, and R_q denotes query instructions. We combine R_s and R_q into a single prompt instruction P_{sq} , as shown in Fig. 3. Subsequently, P_{sq} is then handed over to three ChatGPT roles for processing. The responses from the three different roles are consolidated into L . Then, based on the sub-requirement R'_i , knowledge graph K , and the content list L , we initialize a prompt P_0 . We specify P_0 and K as contextual information, then we interact with ChatGPT using IterativePrompt. Finally, ChatGPT returns the result C_d , where C_d represents the PlantUML scripts for the class diagram or activity diagram.

IterativePrompt as shown in Algorithm 4, takes a prompt instruction P and an error set E_s as input. E_s is a tuple (E, P_u) consisting of an error type E and a correction prompt P_u . The output is C_d . First, we set the maximum iteration count $miCnt$ to 3. If C_d does not satisfy the requirements, we analyze the error type E and check if there is a corresponding error type in E_s . If there exists an error type E , we update P using the correction prompt P_u ; otherwise, we write a new correction prompt to update P . Should C_d satisfy

Algorithm 4 Iterative Prompt.

Input: string prompt P , error set E_s
Output: C_d

```

1: function ITERATIVEPROMPT( $P, E_s$ )
2:    $miCnt \leftarrow 3$ 
3:   for  $i = 0 \rightarrow miCnt$  do
4:      $C_d \leftarrow \text{CHATGPTGEN}(P)$ 
5:     if  $C_d$  is satisfactory then
6:       if  $i > 0$  and  $E_s$  does not contain error type  $E$  related to  $P$  then
7:         add  $(E, P)$  to  $E_s$ 
8:       end if
9:     else
10:      if  $i = miCnt$  then
11:        break
12:      end if
13:      identify error type  $E$  of  $C_d$ 
14:      if  $E$  found in  $E_s$  then
15:         $P \leftarrow E_s.get(E)$ 
16:      else
17:         $P \leftarrow$  a new string prompt for updating  $C_d$ .
18:      end if
19:    end if
20:  end for
21:  if  $C_d$  isn't satisfactory then
22:    manually modify  $C_d$ 
23:  end if
24:  return  $C_d$ 
25: end function

```

the requirements and be obtained through iteration, we add (E, P) to E_s and end the iteration. Should the iteration end and C_d still does not meet the requirements, we manually modify C_d .

There are different sets of E_s for generating class diagrams and activity diagrams. Since the output from ChatGPT may not converge even after multiple rounds of iterations, we set the maximum iteration count $miCnt$ to 3. When $miCnt$ exceeds 3, but the result still isn't satisfactory, we manual correction of the result will be performed.

The UML model generated during the modeling phase is described in text using PlantUML scripts. During the code generation phase, the textual description of the class diagram is used as input for generating SQL scripts and Java class code. Similarly, the textual description of the activity diagram serves as input when generating Java method code.

This detailed process combines the strengths of ChatGPT and UML modeling, ensuring a smooth transition from natural language requirements to Java code.

3.3. Code generation

(1) Generating SQL scripts

During execution, the system requires database access. Therefore, the first step of the code generation phase is to generate the database scripts to build database tables, as outlined in Algorithm 5. C_c represents all the class diagram PlantUML scripts generated during the modeling phase. $C'_c \subseteq C_c$ represents the class diagram PlantUML scripts used in a single process. Assuming that generating all SQL scripts needs to be done in n rounds, where i represents the i^{th} round. Since the later-generated SQL scripts may have relationships with the previously generated ones, when $i > 1$, the prompt P_{cs} should not only include C'_c and the task instruction T , but also the relationship prompt R_l between the expected results of this round and the results generated in the previous round. Finally, ChatGPT return C_d , which represents all the SQL scripts.

(2) Generating Java code

In order to generate Java code, we need to write project information prompt P_s to provide basic project information to ChatGPT. When we describe the basic structure of the project in P_s , it should include the project name, package name, the storage location of relevant classes, programming frameworks and third-party libraries, etc. If system uses programming frameworks and third-party libraries, it is necessary to verify whether ChatGPT is familiar with these contents. If ChatGPT is unfamiliar with these contents, we explain them to ChatGPT by introducing domain knowledge into the context (such as framework introductions, third-party library API documentation, etc.). Generating Java code process consists of three steps: generating configuration files, generating Java classes, and generating business methods.

Generating configuration files: The method for generating configuration files is outlined in Algorithm 6. Firstly, we initialize the error set E_s and the task instruction T , and specify the ChatGPT role. P_s and T are merged into a single prompt instruction P_{pc} . Then, we interact with ChatGPT using IterativePrompt. Finally, ChatGPT returns the result C_d , which represents the generated configuration file code.

Algorithm 5 Generating SQL Scripts.

Input: role instruction I_r , class diagram PlantUML script C_c, C'_c , error set E_s , knowledge graph Cypher script K

Output: SQL script C_d

```

1: function GENERATESQL( $I_r, C_c, C'_c, E_s, K$ )
2:    $n \leftarrow \text{len}(C_c)/\text{len}(C'_c)$ 
3:   CHATGPTGEN( $I_r$ )
4:   CHATGPTGEN( $K$ )
5:   for  $i = 1 \rightarrow n$  do
6:     initialize  $T \leftarrow$  a string prompt for generating SQL script
7:     if  $i = 1$  then
8:        $P_{cs} \leftarrow C'_c + T$ 
9:     else
10:      initialize  $R_i \leftarrow$  a string prompt for describing the relationship between the current result  $C_{di}$  and  $C_{di-1}$ 
11:       $P_{cs} \leftarrow C'_c + T + R_i$ 
12:    end if
13:     $C_{di} \leftarrow \text{INTERACTIVEPROMPT}(P_{cs}, E_s)$ 
14:     $C_d \leftarrow C_d + C_{di}$ 
15:  end for
16:  return  $C_d$ 
17: end function

```

Algorithm 6 Generating Configuraion Files.

Input: role instruction I_r , description about project structure P_s , error set E_s

Output: configuration file code C_d

```

1: function GENERATECONFIGFILES( $I_r, P_s, E_s$ )
2:   initialize  $E_s$ 
3:   initialize  $P_s \leftarrow$  a string prompt for describing project information
4:   CHATGPTGEN( $I_r$ )
5:   CHATGPTGEN( $P_s$ )
6:   initialize  $T \leftarrow$  a string prompt for generating configuration files
7:    $P_{pc} \leftarrow P_s + T$ 
8:    $C_d = \text{ITERATIVEPROMPT}(P_{pc}, E_s)$ 
9:   return  $C_d$ 
10: end function

```

Generating Java class code: Generating Java class code consists of two steps: modifying the class diagram and generating Java classes, as depicted in Algorithm 7. To ensure that the content generated in the modeling phases is not limited to a single programming language and framework, the class diagram generated earlier represents a highly abstract view of software requirements. In order to make the class diagrams fit the framework used by the system, we need to modify the original class diagram to obtain a new one that conforms with the framework. ChatGPT is prompted to adjust the PlantUML script of the original class diagram according to P_s . As a result, it generates a new PlantUML script reflecting the modified class diagram that satisfies P_s . In traditional Model-Driven Development (MDD), specialized code generators are usually tailored for specific projects. However, due to the performance of LLMs in code generation and understanding, LLMs have the potential to serve as code generators in MDD. The specific approach enables us to prompt ChatGPT to generate Java class codes based on the class diagram PlantUML scripts. Since P_s is used to describe the basic information of the project, we can directly generate complete Java class code including imports. However, during the generation process, we prompt ChatGPT to only generate method bodies for classes without the method contents.

Generating business method code: The approach for generating business methods is detailed in Algorithm 8. Firstly, we initialize the error set E_s . Then, we leverage the knowledge graph K and the project information P_s as contextual information. Also, we specify the role instruction I_r to designate ChatGPT's role. Next, we determine if there is a corresponding activity diagram for the business method to be generated (since for simple business methods, we do not use activity diagrams). If an activity diagram exists, we conduct detailed design to convert the activity diagram PlantUML Script into a flowchart to enhance prompt specificity. We proceed to obtain relevant entity class code C_e . Further, we use the zero-shot-CoT technology to initialize the task instruction T , which includes the key phrase "Let's think step by step". This prompts ChatGPT to generate code step by step based on the provided flowchart C_f . We merge C_f , C_e , and T into a single prompt instruction P_{ab} , then use the IterativePrompt method. Finally, ChatGPT returns C_d , representing the generated business method code.

(3) Verifying code correctness

Firstly, we use an IDE to check for compile-time errors in the code generated by ChatGPT. This step involves reviewing the code as it is generated to ensure syntax correctness. Next, we perform unit tests on individual method to determine if they pass the unit tests. Finally, we conduct interface testing to assess whether the interface achieves the correct functionality.

Algorithm 7 Generating Java Class Code.

Input: role instruction I_r , knowledge graph Cypher script K , description about project structure P_s , error set E_s , E'_s , class diagram PlantUML script C_c

Output: Java class code C'_d

```

1: function GENERATEJAVACLASS( $I_r, K, P_s, E_s, E'_s, C_c$ )
2:   initialize  $E_s, E'_s$ 
3:   CHATGPTGEN( $K$ )
4:   CHATGPTGEN( $I_r$ )
5:   CHATGPTGEN( $P_s$ )
6:   step 1: modifying class diagram
7:   define intermediate result  $O$ 
8:   for each class code  $C_i$  in  $C_c$  do
9:      $T \leftarrow$  a string prompt for updating  $C_i$ 
10:     $P_{cc} \leftarrow C_i + T$ 
11:     $C_{di} \leftarrow \text{ITERATIVEPROMPT}(P_{cc}, E_s)$ 
12:     $C_d \leftarrow C_d + C_{di}$ 
13:   end for
14:    $C' = C_d$ 
15:   step 2: generating Java Class Code
16:   for each class  $C'_i$  in  $C'$  do
17:      $T \leftarrow$  a string prompt for generating Java class code
18:      $P_{cj} \leftarrow C'_i + T$ 
19:      $C'_{di} \leftarrow \text{ITERATIVEPROMPT}(P_{cj}, E'_s)$ 
20:      $C'_d \leftarrow C'_d + C'_{di}$ 
21:   end for
22:   return  $C'_d$ 
23: end function

```

Algorithm 8 Generating Business Method Code.

Input: role instruction I_r , knowledge graph Cypher script K , description about project structure P_s , error set E_s , Java entity class code C_e , activity diagram PlantUML script C_a

Output: business method code C_d

```

1: function GENERATEBMCODE( $I_r, K, P_s, E_s, C_e, C_a$ )
2:   initialize  $E_s$ 
3:   CHATGPTGEN( $K$ )
4:   CHATGPTGEN( $I_r$ )
5:   CHATGPTGEN( $P_s$ )
6:   if activity diagram script  $C_a$  exists then
7:     generating the flowchart script and assign it to  $C_f$ .
8:      $P_{ab} \leftarrow P_{ab} + C_f$ 
9:   end if
10:  assign the selected entity class code to  $C_e$ .
11:   $T \leftarrow$  a string prompt for generating business method code
12:   $P_{ab} \leftarrow P_{ab} + C_e + T$ 
13:   $C_d \leftarrow \text{ITERATIVEPROMPT}(P_{ab}, E_s)$ 
14:  return  $C_d$ 
15: end function

```

3.4. Verification

To further verify the correctness of the generated Java code, we employ runtime verification at code level [34] approach as follows:

- (1) We transform a Java code to be verified into an MSVL program M ;
- (2) We translate the negation of property P to be checked into another MSVL program M' ;
- (3) The MSVL program $M \text{ and } M'$ is executed by MSVL compiler MC8.0. If the result is true, a counter example has been found otherwise the $M \text{ and } M'$ needs re-execute with new inputs.

For step (2), we are lucky to use a toolkit P2M developed in our lab. In step (3), we can also use the compiler MC8.0 developed in our lab. Therefore, in the following we focus on how to transform a Java method into an MSVL program (J2M). The processing procedure of J2M is shown in Fig. 4. It first preprocesses the Java code. Then it uses JavaParser [38] to perform lexical and syntax analysis on the processed Java code. If an error occurs, it will return an error message and end the process. Otherwise, it will generate an Abstract Syntax Tree (AST) and then perform code translation. During the code translation process, it will sequentially perform traversal of AST, declaration translation, statement translation, and other translations (such as processing Java classes, collections, etc.). After post-processing the translated code, it will generate the final MSVL program. Fig. 5 shows an example of an insertion sorting algorithm. The code on the left is written in Java language. The code on the right is the MSVL code translated from Java code. As this article mainly focuses on code generation work, we will not go into detail about J2M here.

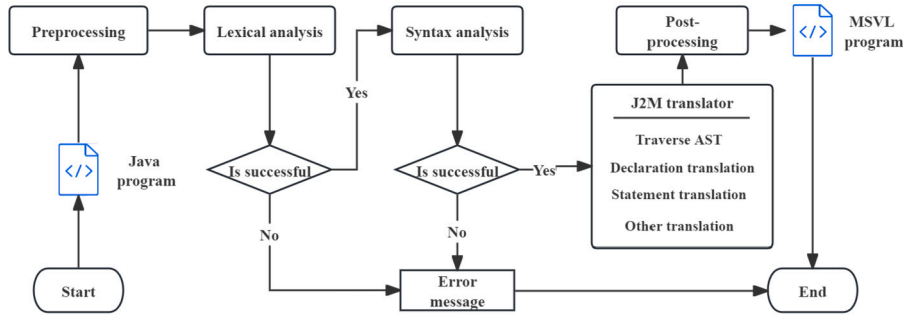


Fig. 4. Processing procedure of J2M.

```

1 package top.java2msvl;
2
3 public class InsertSortExample {
4
5     public static void main(String[] args) {
6         int[] array = new int[]{3, 1, 5, 9,
7         6, 7, 2, 4, 8, 0};
8         for (int a : array) {
9             System.out.print(a + ",");
10        }
11        System.out.println("\n");
12        insertSort(array);
13    }
14    public static void insertSort(int[]
15    array) {
16        int n = array.length;
17        for (int i = 1; i < n; i++) {
18            for (int j = i; j > 0 &&
19            array[j] < array[j - 1]; j--) {
20                int temp = array[j];
21                array[j] = array[j - 1];
22                array[j - 1] = temp;
23            }
24            for (int a : array) {
25                System.out.print(a + ",");
26            }
27            System.out.println("\n");
28        }
29    }
30 }

```

```

1 frame(array, m_i, m_j, m_temp) and (
2     int array[10]<={3,1,5,9,6,7,2,4,8,0} and
3     skip;
4     int m_i and m_i:=1;
5     int m_j and m_j:=0;
6     int m_temp and m_temp:=0;
7     while(m_j<10){
8         output(array[m_j],",") and skip;
9         m_j:=m_j+1
10    };
11    output("\n","\n") and skip;
12    while(m_i<10)
13    {
14        m_j:=m_i;
15        while(m_j>0 AND array[m_j]<array[m_j-
16        1])
17        {
18            m_temp:=array[m_j];
19            array[m_j]:=array[m_j-1];
20            array[m_j-1]:=m_temp;
21            m_j:=m_j-1
22        };
23        m_j:=0;
24        while(m_j<10)
25        {
26            output(array[m_j],",") and skip;
27            m_j:=m_j+1
28        };
29        output("\n","\n") and skip;
30        m_i:=m_i+1
31    }
32 )

```

Fig. 5. A Java2MSVL Example.

4. Application

This section will introduce the application of our proposed approach in a practical software system. Our approach has been applied in the development process of a practical sales management system for a building materials sales enterprise. The system is a web software system developed in Java, totaling around 20,000 lines of code. Due to space limitations, this chapter is unable to show the entire system development process. We will try our best to demonstrate some key steps.

4.1. Requirement analysis

(1) Requirement Decomposition

We use the requirement decomposition approach described in section 3.1 to guide ChatGPT in decomposing user requirement R . As shown in Fig. 6, the left side shows user requirement R , and the right side shows the final sub-requirement results. Due to its length, we have omitted some information from R . In order to save tokens, we use the prompt L_i to limit the output of ChatGPT when it is not required to generate output. The principle of requirement decomposition P_d and output limitation instruction L_i are shown in Fig. 7. Among them, ChatGPT generated unsatisfactory results when decomposing sub requirements. For example, it includes prepayments, sales orders, and product management blocks in the financial management block. At this point, we prompted it to further subdivide the financial management block based on R , and ultimately obtained the desired result.

(2) Generating Knowledge Graph

We use Algorithm 2 to construct a knowledge graph. Fig. 8 shows this process and the final result. Firstly, we send role instructions to ChatGPT to focus on requirement analysis, information extraction, and generating Cypher script. Secondly, we send requirement R to ChatGPT as prior knowledge. Thirdly, we prompt ChatGPT to extract information from R . Meanwhile, we utilize the (E, R_i, A) triples to standardize the output of ChatGPT. When we review the results, we find that ChatGPT often misses a lot of entity and rela-

A building materials company needs to customize a web application called "Building Materials Sales System". This web application is designed for internal personnel within the company to simplify financial management and shipping processes. The main business of the company is to produce hollow bricks and then sell them to customers.

The following content is about the business that web applications need to implement.

(1) There are four types of roles using this application, namely senior manager, general manager, finance officer, and shipping officer.

(2) Customers are related enterprises that purchase hollow bricks, and usually one customer will include multiple projects. Meanwhile, there are also some customers who make personal purchases, which we refer to as retail customers. Customer information and project information need to be recorded, while retail customer information does not need to be recorded.

.....

(9) When system users perform some risk operations, they need to record operation logs, and the system needs to record log information.

Next are some business details.

(1) Customers have two settlement methods (advance payment and monthly settlement).

.....

Finally, there are some non functional requirements.

(1) System users must log in to the system before they can manipulate data.

(2) The average response time of the interface cannot exceed 1000 milliseconds.

(3) The system should be easy to expand after development is completed.

.....

R1. User Management Block:

Responsible for managing user roles, authentication, and authorization.

R2. Customer Management Block:

Responsible for managing customer information, including enterprises and retail customers.

Data includes customer details (name, contact info), and retail customer status.

Functions include adding, updating, and deleting customer records.

R3. Project Management Block:

Responsible for managing project information associated with enterprise customers.

Data includes project details, such as project name, location, and status.

Functions include adding, updating, and deleting project records.

R4. Prepayment Management Block:

Responsible for managing prepayments made by customers.

Data includes prepayment amounts and customer balances.

Functions include recording prepayments and updating customer balances.

.....

R8. Inventory Management Block:

Responsible for managing inventory levels of hollow bricks.

Data includes inventory details (types, quantities) and pallet information.

Functions include updating inventory levels, recording incoming shipments, and managing pallets.

R9. Pallet Management Block:

Responsible for managing information about pallets used for loading hollow bricks.

Data includes pallet details (ID, condition, location).

Functions include adding, updating, and deleting pallet records.

Fig. 6. Requirement decomposition in the practical system.

Li : You don't need to provide any explanation for it, just use it as prior knowledge in subsequent interactions.

Pd : According to the data management requirements of the system, decompose the requirements into different data blocks. Each block is responsible for managing specific types of data, including data input, data storage, data modification, and data querying. Simultaneously following the principles of high cohesion, low coupling, single responsibility, and traceability.

Fig. 7. Prompt about L_i and P_d .

tionship information. For example, entity information such as company and account information, as well as production relationships between companies and products. To do that, we provide examples of missing entities and relationships in the prompt, prompting ChatGPT to continue mining information from R based on the examples. Finally, we obtain a satisfactory triplet (E, R_i, A) .

Next, we prompt ChatGPT based on (E, R_i, A) generate Cypher scripts. We find that after using a triplet as prompt information, the Cypher script generated by ChatGPT is also written by first creating entity nodes, then creating relationship edges, and finally filling in attributes for the entities. The generated Cypher script matches this triplet. However, there are also issues in the first generated results. For example, there are syntax errors in the script and the creation of some entities is missing from the script. The syntax error is reflected in our execution of ChatGPT's first generated script in Neo4j, resulting in the generation of twice the number of nodes. Then we prompt ChatGPT to use a separate CREATE statement for each node and edge. And at the same time, we remind it to supplement the missing node creation statement. Finally, we obtain the correct result. Fig. 9 shows the visualization results generated after executing the script in the Neo4j database.

(3) Refining Requirement

For each sub-requirement R_i , we will refine it. We consider all the data, functionality, conditions, and some nonfunctional requirements it requires. In the end, we obtained R'_i according to our proposed approach. It includes three parts: data, functionality, and constraints. Fig. 10 illustrates the partitioning format of R'_i through an example. This shows the data that the customer block needs to record, the functionalities that need to be implemented, and the constraints that need to be met during the design.

4.2. Generating UML diagram

(1) Generating Class Diagram

We model R'_i by combining multiple modules within the token limits of ChatGPT. We follow Algorithm 3 to prompt ChatGPT to generate PlantUML scripts corresponding to the class diagram based on R'_i . The prompt we send to ChatGPT and the final output result are illustrated in Fig. 11.

(2) Generating Activity Diagram

For the functional portion of R'_i , we prompt ChatGPT to generate PlantUML scripts for the activity diagram of a specific functionality. When generating the activity diagram, our interaction with ChatGPT is depicted in Fig. 12.

During the initial interaction, we refrain from providing ChatGPT with the specific activity flow of the functionality to leverage its creativity. Subsequently, in the following rounds of interaction, we iteratively modify the graphics generated in the first round by offering more detailed descriptions. The visual representation of the final activity diagram is shown in Fig. 14.

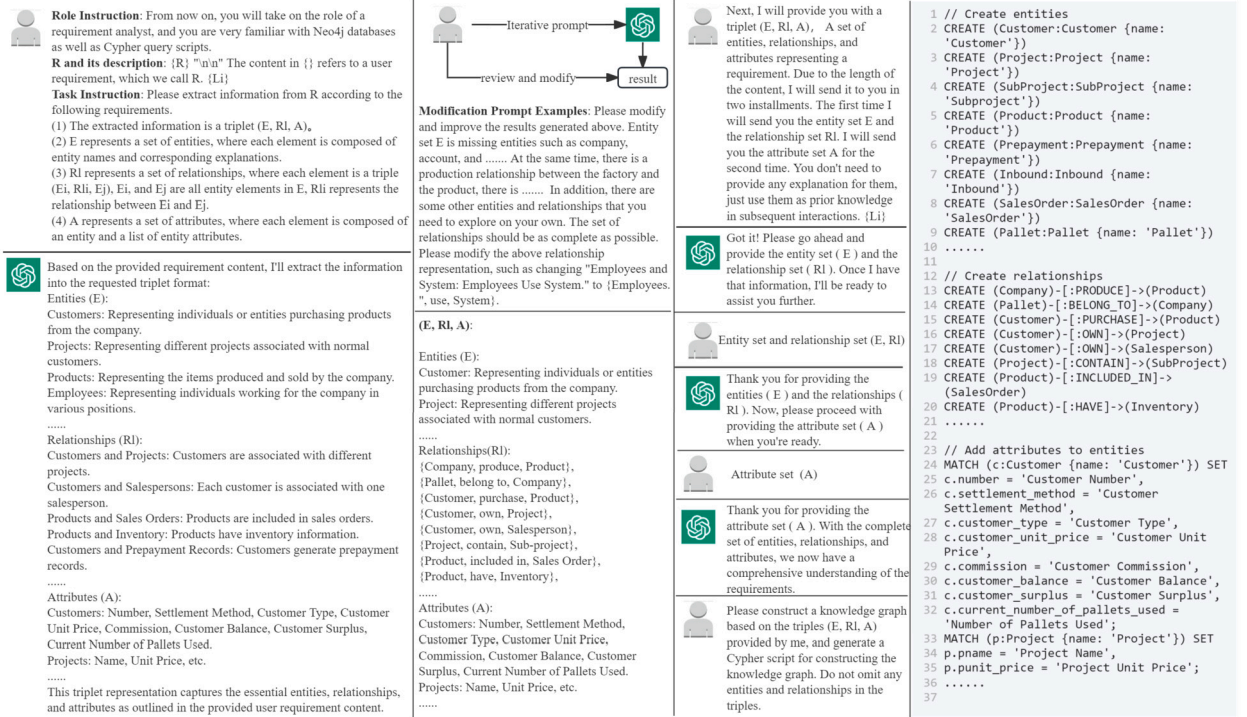
Fig. 8. Extracting message from user requirement R and generating Cypher scripts based on triple (E, R_i, A) .

Fig. 9. A part of the knowledge graph based R.

4.3. Generating Java code

(1) Generating SQL script

We use Algorithm 5 to generate database scripts. The input PlantUML scripts and Cypher scripts are formatted using Markdown code block notation, enclosed in "```. Throughout this process, discrepancies may arise between the generated data table field types and the actual field types we require. In such cases, we ask ChatGPT to modify these mismatched field types. Additionally, other

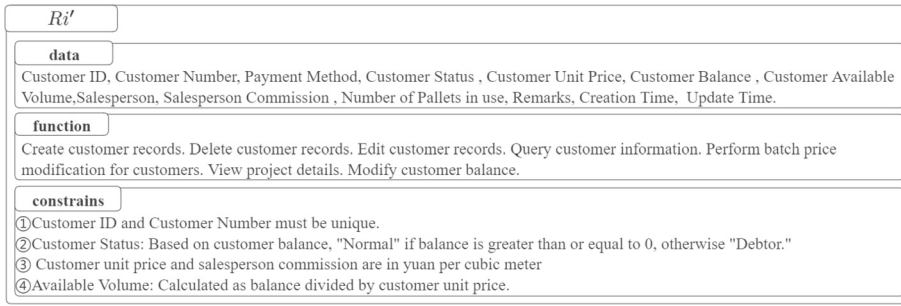
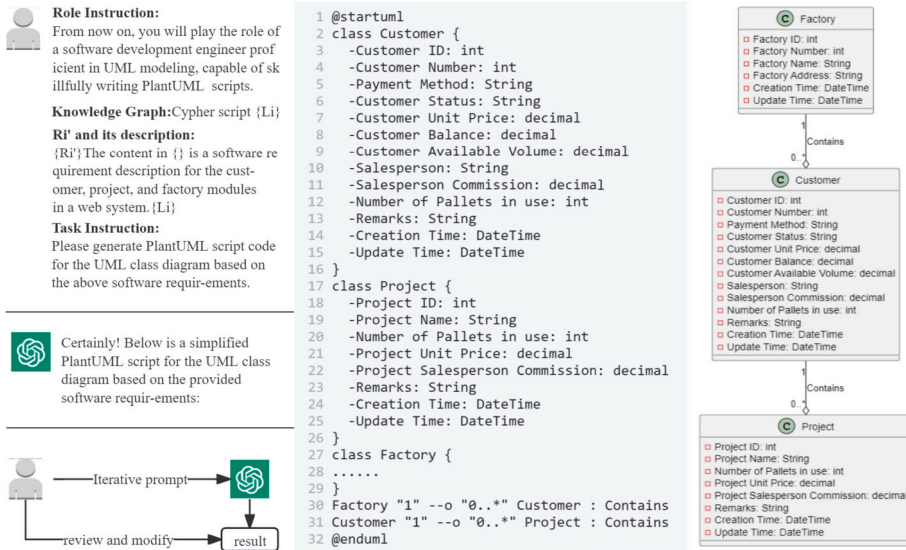
Fig. 10. Example of R_i' .

Fig. 11. Generating class diagrams with ChatGPT.

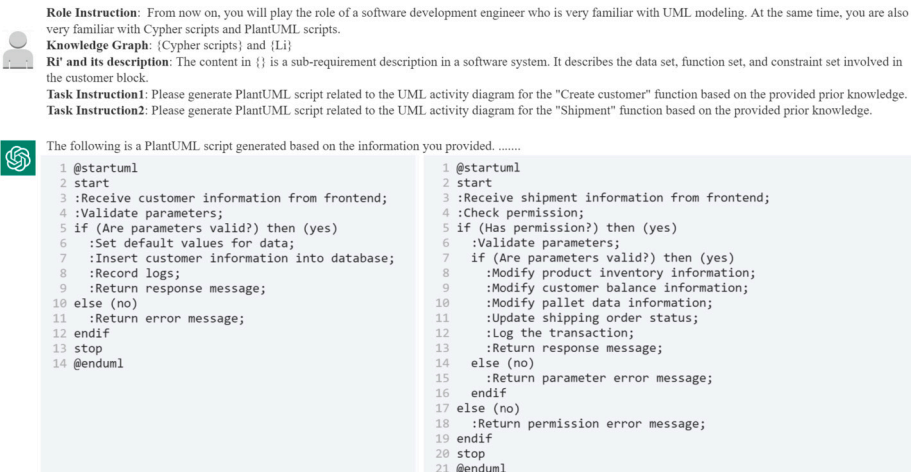


Fig. 12. Generating activity diagrams with ChatGPT.

Role Instruction: From now on, you will act as a software development engineer proficient in PlantUML script, Cypher script, and SQL script.

Class diagrams, Knowledge Graph, and their descriptions:

...Plantuml... The above code is generated based on a software requirement for PlantUML, referred to as Code X.

...cypher... The above code is generated based on a software requirement for Cypher, referred to as Code Y.

Task Instruction: Based on Code X and code Y, generate SQL script code suitable for a MySQL database.

Certainly, based on Code X and Code Y, here's a simplified example of SQL script for creating tables in a MySQL database:

```

1 CREATE TABLE customer (
2   customer_id INT PRIMARY KEY AUTO_INCREMENT,
3   customer_number INT UNIQUE,
4   payment_method VARCHAR(255),
5   customer_status VARCHAR(255),
6   customer_unit_price DECIMAL(10, 3),
7   customer_balance DECIMAL(10, 3),
8   customer_available_volume DECIMAL(10, 3),
9   salesperson VARCHAR(255),
10  salesperson_commission DECIMAL(10, 3),
11  number_of_pallets_in_use INT,
12  remarks VARCHAR(500),
13  creation_time DATETIME,
14  update_time DATETIME,
15  factory_id INT
16 ) ENGINE=InnoDB;
17 .....

1 CREATE TABLE log (
2   log_id INT PRIMARY KEY AUTO_INCREMENT,
3   creation_time DATETIME,
4   operator_name VARCHAR(255),
5   unit_price_before_modification DECIMAL(10, 3),
6   log_content VARCHAR(500),
7   factory_id INT
8 ) ENGINE=InnoDB;
9 .....
10 ALTER TABLE customer
11 ADD CONSTRAINT fk_customer_factory
12 FOREIGN KEY (factory_id) REFERENCES
13   factory(factory_id);
14 .....
15 ALTER TABLE log
16 ADD CONSTRAINT fk_log_factory
17 FOREIGN KEY (factory_id) REFERENCES
18   factory(factory_id);
19 .....

```

Fig. 13. Generating SQL script with ChatGPT.

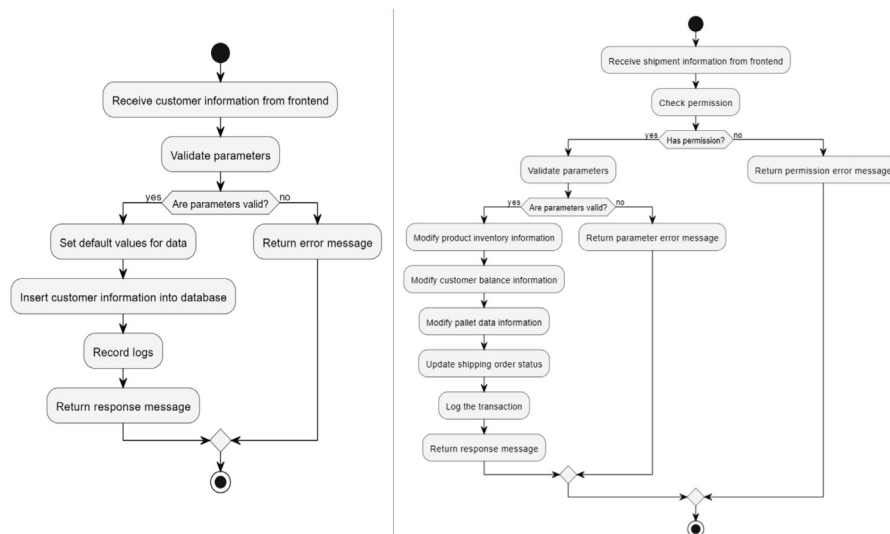


Fig. 14. Visual results of the activity diagram.

errors may occur, such as incorrect content ordering in the generated script causing database errors. Our solution involves sending the error information to ChatGPT and using its modified script code. An illustrative example of our interaction with ChatGPT during the SQL script generation phase is shown in Fig. 13.

(2) Modifying UML class diagram

We need to adapt the class diagram created during the modeling phase to fit the programming framework used in the project. To enable ChatGPT to assist with modifying the class diagram, we must provide a prompt that describe the project details. We refer to this prompt as Ps. Fig. 15 shows the content of Ps and a part of a modified class diagram example.

(3) Generating Java class code

We extracted each class from the modified class diagram and sequentially sent the PlantUML scripts for these classes to ChatGPT. These scripts are surrounded using the “...” format. Our goal is to generate Java class code for each class, forming the initial code framework. Fig. 16 illustrates an example of our interaction with ChatGPT, demonstrating the generation process for the “CustomerServiceImpl” class.

(4) Generating business method code

After completing the Java class code, we gradually fill in the empty method bodies in the code. During the interaction, we first provide ChatGPT with the relevant entity class code for the method, the corresponding flowchart script, and Cypher script as prior knowledge for generating business method code. A flowchart is transformed from an activity diagram. For example, when creating a customer, parameter validation is required. In the flowchart, parameter verification is divided into three steps: verifying customer number, verifying customer unit price, and verifying customer uniqueness. Then, we present ChatGPT with the method body that needs to be generated and other relevant content of the containing class (including package import code and the class name). We

P_s The entire project uses Spring Boot as the backend framework, with MyBatis-Plus as the data persistence layer framework. The backend project structure is organized into layers, including packages such as entity, mapper, service, service.impl, and controller. The mapper.xml files are stored in the mapper directory under the resources folder. Please modify the above class diagram PlantUML scripts so that each class has a corresponding entity class, mapper interface, service interface, serviceImpl implementation class, and controller class. The entity class should directly use the entity name as the class name, while other classes should be named as the entity name followed by the respective layer name. For example, the mapper interface for the customer class would be named CustomerMapper. Implement the corresponding interfaces and methods in different layers according to the provided software requirements.

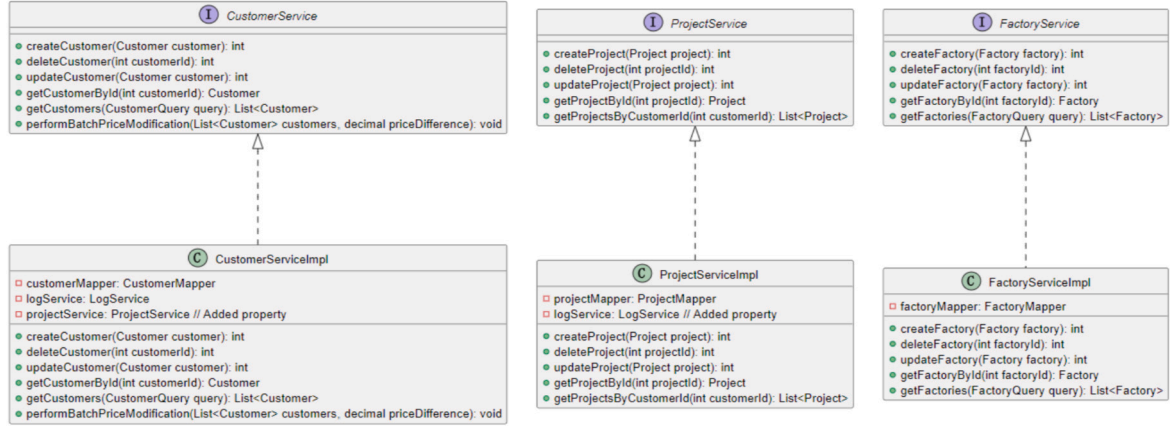


Fig. 15. The content of P_s and a part of a modified class diagram example.



Role Instruction: From now on, you will act as a software development engineer proficient in Java, Plantuml and Cypher language.

Ps: I am planning to build a Java backend project using the Spring Boot framework. The project is named 'sales-chatgpt-test1', and its main structure is described as follows:

{Ps}, Please consider the information I provided above as prior knowledge for use in upcoming questions. You don't need to explain it; just respond with 'Okay.'

Plantuml And Cypher: new class diagram, knowledge graph

Task Instruction:

Please generate Java class code corresponding to the above PlantUML script. Add the @Service annotation to each generated class, and make them extend the ServiceImpl class from MyBatisPlus. Use the @Autowired annotation for obtaining certain properties within the classes. There's no need to write any content inside the methods; if a method requires a return value, simply return null. Ensure that all necessary packages are imported in the code.

Modify: Please modify the above code. Carefully inspect the code for any missing imports, and add the necessary imports, such as BigDecimal and the Service Impl from MyBatis-Plus.



Certainly! Below is the generated Java class code corresponding to the provided PlantUML script:

```
1 package cn.edu.xidian.saleschatgpttest1.service.impl;
2 import cn.edu.xidian.saleschatgpttest1.entity.Customer;
3 import cn.edu.xidian.saleschatgpttest1.mapper.CustomerMapper;
4 import cn.edu.xidian.saleschatgpttest1.query.CustomerQuery;
5 .....
6 import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Service;
9 import java.math.BigDecimal;
10 import java.util.List;
11 @Service
12 public class CustomerServiceImpl extends ServiceImpl<CustomerMapper, Customer> implements CustomerService {
13     @Autowired
14     private CustomerMapper customerMapper;
15     public int createCustomer(Customer customer) {return 0;}
16     public int deleteCustomer(int customerId) {return 0;}
17     public int updateCustomer(Customer customer) {return 0;}
18     public Customer getCustomerById(int customerId) {return null;}
19     public List<Customer> getCustomers(CustomerQuery query) {return null;}
20     public void performBatchPriceModification(List<Customer> customers, BigDecimal priceDifference) {
21         // Method implementation here
22     }
23 }
```

Fig. 16. Generating Java class code with ChatGPT.

prompt ChatGPT to generate the code step by step according to the steps in the flowchart. Fig. 17 illustrates the interaction process when generating the business method code for creating a customer.

The business method code generated by ChatGPT is not always satisfactory. For example, the code generated included unknown method names. Then, we remind ChatGPT in the prompt that it can call methods from third-party libraries to achieve the same functionality.

4.4. Verifying code correctness

After the generated code does not report any errors in the IDE, we use unit testing and interface testing methods to test the generated code. Then we further verify the correctness of the generated code using a runtime verification at code level approach. We use J2M to translate Java code into MSVL programs. Fig. 18 shows the transformation we performed in the practical business code. We convert the Customer class in Java code into the Customer structure in MSVL. The String in the class has been replaced with a char array in the structure, and each property in the structure is connected using the and symbol. We use CusNode to simulate



Role Instruction: From now on, you will act as a software development engineer proficient in PlantUML script, Cypher script and Java language.

Plantuml: ``Flowchart code for creating a customer.'`

Knowledge Graph: ``Cypher code``

Java: ``Customer entity class code``

Task Instruction: Please consider the above code as prior knowledge for the upcoming interactions. Based on the prior knowledge provided earlier, Write the 'createCustomer' method in the following code according to the steps in the PlantUML activity diagram.



```
1 @Override
2 public int createCustomer(Customer customer) {
3     if (customer.getCustomerNumber() == null || customer.getCustomerNumber().isEmpty()) {
4         return -1;
5     }
6     if (customer.getCustomerUnitPrice().compareTo(new BigDecimal(0)) < 0) {
7         return -1;
8     }
9     LambdaQueryWrapper<Customer> queryWrapper = new LambdaQueryWrapper<>();
10    queryWrapper.eq(Customer::getCustomerNumber, customer.getCustomerNumber());
11    if (customerMapper.selectCount(queryWrapper) == 0) {
12        customer.setCustomerBalance(BigDecimal.ZERO);
13        customer.setCustomerAvailableVolume(customer.getCustomerBalance().divide(customer.getCustomerUnitPrice(), 3, RoundingMode.HALF_UP));
14        customer.setCustomerStatus("Normal");
15        customer.setNumberOfPalletsInUse(0);
16        LocalDateTime currentTime = LocalDateTime.now();
17        customer.setCreationTime(currentTime);
18        customer.setUpdateTime(currentTime);
19        customerMapper.insert(customer);
20        logService.createLog(customer);
21        return 1;
22    } else {
23        return -1;
24    }
25 }
26
27 // Other methods...
```

Fig. 17. Generating business method code with ChatGPT.

<pre>1 </ 2 define p1:pCusUnitPrice>0; 3 define p2:pProjUnitPrice>0; 4 alw(p1 and p2) 5 /> 6 7 8 9 struct Customer{ 10 int customerId and 11 char customerNumber[255] and 12 char paymentMethod[255] and 13 char customerStatus[255] and 14 double customerUnitPrice and 15 double customerBalance and 16 double customerAvailableVolume and 17 char salesperson[255] and 18 double salespersonCommission and 19 int numberOfPalletsInUse and 20 char remarks[500] and 21 int factoryId 22 }; 23 24 25 struct CusNode{ 26 struct Customer cus and 27 struct Node *nextCusNode 28 }; 29 30 31 32 33</pre>	<pre>1 2 function performBatchUpdatePrice(struct CusNode *de_cusNode, double c_priceDifference) 3 { 4 frame(de_curCusUnitPrice) and (5 int de_curCusUnitPrice==0 and skip; 6 while(de_cusNode!=NULL) 7 { 8 de_curCusUnitPrice:=de_cusNode->cus.customerUnitPrice; 9 pCusUnitPrice:=de_curCusUnitPrice; 10 de_curCusUnitPrice:=de_curCusUnitPrice-c_priceDifference; 11 if(de_curCusUnitPrice<0) then 12 { 13 // performBatchUpdatePrice rollback logic 14 15 } 16 else 17 { 18 de_cusNode->cus.customerUnitPrice:=de_curCusUnitPrice 19 }; 20 struct Project *curProject:=NULL and skip; 21 getProjectByCusId(&curProject, de_cusNode->cus.customerId); 22 while(curProject!=NULL) 23 { 24 decreaseProjectUnitPrice(curProject, c_priceDifference); 25 curProject:=curProject->nextProjNode 26 }; 27 de_cusNode:=de_cusNode->nextCusNode; 28 pCusUnitPrice:=de_curCusUnitPrice 29 }; 30 31 } 32 }; 33</pre>
--	--

Fig. 18. PPTL formula and MSVL code.

a Customer linked list. The “performBatchUpdateprice” method represents batch modification of unit prices, which is MSVL code translated from a method in the Java system. In addition, there are many other similar conversions.

When we use $MC_{8.0}$ to verify code, we need to use $</.../>$ in the .p file to define the PPTL formula, as shown in the upper left corner of Fig. 18. At the same time, we write the MSVL program in a .m file. Then, when we use the $MC_{8.0}$ verifier to execute the program, the verifier will concatenate the content in the .p file with the content in the .m file to form a program for execution. Finally, we obtain the verification result.

For example, in our code, there is a business method for batch modifying unit prices. Before and after modifying the unit price, both the customer unit price and project unit price are non negative. So, we use pCusUnitprice to represent customer unit price and pProjUnitprice to represent project unit price. We assign non negative values to these two variables in the initial state of the MSVL program. Then, when executing the method, assign values to these two variables sequentially using the unit price before and after modification. So, we can define p1 as $pCusUnitprice > 0$ and p2 as $pProjUnitprice > 0$ in the PPTL formula, and the



```

x64\Release\MCS8.0.exe
$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
ThreadPool init success!!!

Verification Result: Valid!!!
release ThreadPool success!!!
The runtime verification time is: 188ms
All time is: 1266ms

```

Fig. 19. Code verification result.

property to be verified is $\text{alw}(p_1 \text{ and } p_2)$, indicating that p_1 and p_2 always hold. Fig. 18 and Fig. 19 shows the PPTL formula, MSVL code and verification results. The verification result shows “Valid”.

4.5. Discussion

Firstly, we have ChatGPT inspect the generated code. It always provides some optimization suggestions. This means that although we ensure the functional correctness of the program, there may still be potential issues in the code that require further review and modification, which will be the focus of our subsequent work. Secondly, the proposed approach can be executed in parallel for modules with low coupling. Thirdly, if we encounter the need for modifications while generating initial framework code or business method code, it is crucial to ensure simultaneous adjustments to the UML diagrams generated in the modeling phase to maintain consistency between the model and the final Java code.

5. Related work

Code generation based on natural language descriptions has long been a focal area of interest for software engineering researchers. Recently, the advent of LLMs has brought many opportunities for code generation research. General-purpose LLMs such as Llama [39], Llama2 [4], GPT-J [40], GPT-Neo [41], GPT-3 [28], ChatGPT [2], GPT-4 [3], etc. have shown good performance in code generation tasks. There is a lot of code generation work around the LLMs. Researchers have developed LLMs specifically designed for code generation, such as AlphaCode [42], StarCode [43], etc. Some researchers have fine-tuned the general-purpose LLMs and developed successful code generation models such as Codex [9], Code Llama [16], and CodeGen [44]. Nevertheless, these works focus more on general code generation.

Unlike general code generation tasks, this paper focuses on large-scale Java code generation in practical business scenarios. This belongs to the specific domains code generation research. In this regard, Liu et al. [17] and Zhang et al. [45] conduct research on code generation combining with code repositories. Dong et al. [21] and Ishibashi et al. [46] propose the Self Collaboration framework and Self Organized Agents framework for code generation of complex tasks, respectively. Chen et al. [18] conduct empirical research on the code generation of LLMs in specific domains (web domains) and integrated API knowledge into the code generation process.

The performance of a large model highly depends on the quality of the prompts used. This leads to prompt engineering. In this regard, White et al. [26] construct a catalog of prompt patterns and explored some prompt patterns for SE in subsequent work [47]. Brown et al. [28] demonstrate excellent few-shot learning ability of GPT-3, which has attracted the interest of many researchers. Some researchers explore better prompt strategies to unleash the reasoning ability of LLMs. Wei et al. [29] propose chain-of-thought. Wang et al. [31] improve chain-of-thought via self-consistency. Li et al. [48] propose DIVERSE (Diverse Verifier on Reasoning Step). Kojima et al. [27] propose that by designing prompts to guide LLMs to reason step by step, LLMs can become excellent zero-shot reasoners. Liu et al. [15] guide ChatGPT to generate code through carefully designed prompts, ultimately greatly improving generation performance. Inspired by these studies, we utilize some excellent prompt engineering techniques in this paper.

The code generation task in practical business scenarios is often more complex, and the inherent ambiguity of natural language poses great challenges to complex software design. Some researchers attempt to introduce models between natural language and code to reduce ambiguity in natural language. R.Sadik et al. [22] propose an agile MDD method using GPT-4 enhancement, combining UML, OCL, and FIPA ontology. Petrovic et al. [49] propose a code generation method that utilizes LLMs to convert user requirements into Ecore model instances, then performs consistency checks using OCL, and finally generates code using other LLMs. This paper draws on the experience of predecessors and uses UML class diagrams and activity diagrams to semi formally express requirements in the code generation process. Meanwhile, considering that the hallucination problem in LLMs is due to their inability to refer to specific sources of knowledge. We draw on the Graph-of-Thought [50] and Graph-Chain-of-Thought [51] technologies proposed in the field of knowledge graphs. These technologies can enable LLMs to infer from external knowledge and reduce hallucination problems. We also utilize techniques similar to reasoning on knowledge graphs in this paper to assist us in using ChatGPT for requirement analysis, modeling, and code generation.

In LLMs code generation, evaluating the code correctness is equally important as the code generation itself. Pass@k is proposed in [9] and is currently used in many studies. Some studies [15,17] also use metrics such as BLEU and CodeBLEU to compare the generated code with template code for evaluation. However, in practical software development, the code is often complex and may call some external functions, without template code as a reference. In [22], the author evaluates the correctness of the code by comparing the behavior of the generated code to the original sequence diagram. In this paper, we use formal verification methods. We use the runtime verification at code level [34] approach to verify the correctness of the code generated by LLMs.

6. Conclusion

In this paper, we propose Xd-CodeGen, a novel approach that prompts ChatGPT to generate Java code using iterative prompting. It divides the process of Java code generation into four phases: requirement analysis, modeling, code generation, and code verification. The proposed approach has been successfully applied to a practical project development. It demonstrates usefulness and effectiveness of the new approach. Further, it expands the scope of code generation and shows the potential of LLMs in software development. In the future, we will further study more complex programming scenarios and improve the generative performance of LLMs in requirement analysis, modeling, and code generation phases.

CRedit authorship contribution statement

Zelong Zhao: Writing – original draft, Software. **Nan Zhang:** Supervision, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Bin Yu:** Supervision. **Zhenhua Duan:** Writing – original draft, Supervision, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems, 2017*, pp. 5998–6008.
- [2] OpenAI, Chatgpt: optimizing language models for dialogue, <https://openai.com/blog/chatgpt/>, 2022.
- [3] OpenAI, GPT-4 technical report, CoRR, arXiv:2303.08774 [abs], 2023.
- [4] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., Llama 2: open foundation and fine-tuned chat models, CoRR, arXiv:2307.09288 [abs], 2023.
- [5] C. Shen, L. Cheng, X. Nguyen, Y. You, L. Bing, Large language models are not yet human-level evaluators for abstractive summarization, in: *Findings of the Association for Computational Linguistics: EMNLP 2023*, Association for Computational Linguistics, 2023, pp. 4215–4233.
- [6] W. Jiao, W. Wang, J. Huang, X. Wang, Z. Tu, Is chatgpt A good translator? A preliminary study, CoRR, arXiv:2301.08745 [abs], 2023.
- [7] S. Frieder, L. Pinchetti, A. Chevalier, R. Griffiths, T. Salvatori, T. Lukasiewicz, P. Petersen, J. Berner, Mathematical capabilities of chatgpt, in: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems, 2023*.
- [8] D. Zan, B. Chen, F. Zhang, D. Lu, B. Wu, B. Guan, Y. Wang, J. Lou, Large language models meet nl2code: a survey, in: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 7443–7464.
- [9] M. Chen, J. Twarek, H. Jun, Q. Yuan, H.P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, CoRR, arXiv:2107.03374 [abs], 2021.
- [10] A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, M.S. Aktar, T. Mikkonen, Towards human-bot collaborative software architecting with chatgpt, in: *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, ACM, 2023, pp. 279–285.
- [11] S. Feng, S. Ma, J. Yu, C. Chen, T. Zhou, Y. Zhen, Auto-icon: an automated code generation tool for icon designs assisting in UI development, in: *IUI '21: 26th International Conference on Intelligent User Interfaces*, ACM, 2021, pp. 59–69.
- [12] Q. Guo, J. Cao, X. Xie, S. Liu, X. Li, B. Chen, X. Peng, Exploring the potential of chatgpt in automated code refinement: an empirical study, in: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, ACM, 2024, pp. 34:1–34:13.
- [13] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, H. Li, Prompt-enhanced software vulnerability detection using chatgpt, in: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, ACM, 2024, pp. 276–277.
- [14] J.Y. Khan, G. Uddin, Automatic code documentation generation using GPT-3, in: *37th IEEE/ACM International Conference on Automated Software Engineering*, ACM, 2022, pp. 174:1–174:6.
- [15] C. Liu, X. Bao, H. Zhang, N. Zhang, H. Hu, X. Zhang, M. Yan, Improving chatgpt prompt for code generation, CoRR, arXiv:2305.08360 [abs], 2023.
- [16] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X.E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al., Code llama: open foundation models for code, CoRR, arXiv:2308.12950 [abs], 2023.
- [17] M. Liu, T. Yang, Y. Lou, X. Du, Y. Wang, X. Peng, Codegen4libs: a two-stage approach for library-oriented code generation, in: *38th IEEE/ACM International Conference on Automated Software Engineering*, IEEE, 2023, pp. 434–445.
- [18] M. Chen, H. Zhang, C. Wan, Z. Wei, Y. Xu, J. Wang, X. Gu, On the effectiveness of large language models in domain-specific code generation, CoRR, arXiv:2312.01639 [abs], 2023.
- [19] J. Austin, A. Odena, M.I. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C.J. Cai, M. Terry, Q.V. Le, C. Sutton, Program synthesis with large language models, CoRR, arXiv:2108.07732 [abs], 2021.
- [20] J. Cámara, J. Troya, L. Burgeño, A. Vallecillo, On the assessment of generative AI in modeling tasks: an experience report with chatgpt and UML, *Softw. Syst. Model.* 22 (3) (2023) 781–793.
- [21] Y. Dong, X. Jiang, Z. Jin, G. Li, Self-collaboration code generation via chatgpt, CoRR, arXiv:2304.07590 [abs], 2023.
- [22] A.R. Sadiq, S. Brulin, M. Olhofer, Coding by design: GPT-4 empowers agile model driven development, in: *Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering*, SCITEPRESS, 2024, pp. 149–156.
- [23] M. Monteiro, B.C. Branco, S. Silvestre, G. Avelino, M.T. Valente, End-to-end software construction using chatgpt: an experience report, CoRR, arXiv:2310.14843 [abs], 2023.
- [24] P.S. Lawrence, *Software Engineering: Theory and Practice*, Pearson Education India, 1998.
- [25] W.B. Knox, P. Stone, Augmenting reinforcement learning with human feedback, in: *ICML 2011 Workshop on New Developments in Imitation Learning (July 2011)*, vol. 855, 2011.
- [26] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, D.C. Schmidt, A prompt pattern catalog to enhance prompt engineering with chatgpt, CoRR, arXiv:2302.11382 [abs], 2023.
- [27] T. Kojima, S.S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa, Large language models are zero-shot reasoners, in: *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems, 2022*, pp. 22199–22213.

- [28] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, in: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems*, 2020, pp. 1877–1901.
- [29] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E.H. Chi, Q.V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems*, 2022, pp. 24824–24837.
- [30] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, L. Zettlemoyer, Rethinking the role of demonstrations: what makes in-context learning work?, in: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2022, pp. 11048–11064.
- [31] X. Wang, J. Wei, D. Schuurmans, Q.V. Le, E.H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: *The Eleventh International Conference on Learning Representations*, OpenReview.net, 2023.
- [32] Z. Duan, C. Tian, L. Zhang, A decision procedure for propositional projection temporal logic with infinite models, *Acta Inform.* 45 (2008) 43–78.
- [33] Z. Duan, N. Zhang, A complete axiomatization of propositional projection temporal logic, in: *2nd IFIP/IEEE International Symposium on Theoretical Aspects of Software Engineering*, 2008, pp. 271–278.
- [34] M. Wang, C. Tian, N. Zhang, Z. Duan, Verifying full regular temporal properties of programs via dynamic program execution, *IEEE Trans. Reliab.* 68 (3) (2019) 1101–1116.
- [35] K. Yang, Z. Duan, C. Tian, N. Zhang, A compiler for MSVL and its applications, *Theor. Comput. Sci.* 749 (2018) 2–16.
- [36] M. Wang, C. Tian, N. Zhang, Z. Duan, C. Yao, Translating xd-c programs to MSVL programs, *Theor. Comput. Sci.* 809 (2020) 430–465.
- [37] Plantuml, <https://github.com/plantuml/plantuml>.
- [38] Javaparser, <https://github.com/javaparser>.
- [39] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: open and efficient foundation language models, *CoRR*, arXiv:2302.13971 [abs], 2023.
- [40] B. Wang, A. Komatsuzaki, Gpt-j-6b: A 6 Billion Parameter Autoregressive Language Model, 2021.
- [41] S. Black, L. Gao, P. Wang, C. Leahy, S. Biderman, Gpt-neo: large scale autoregressivetouvron2023llama language modeling with mesh-tensorflow. If you use this software, please cite it using these metadata 58 (2021) 2.
- [42] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al., Competition-level code generation with alphacode, *Science* 378 (6624) (2022) 1092–1097.
- [43] R. Li, L.B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al., Starcoder: may the source be with you!, *CoRR*, arXiv:2305.06161 [abs], 2023.
- [44] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, C. Xiong, Codegen: an open large language model for code with multi-turn program synthesis, in: *The Eleventh International Conference on Learning Representations*, OpenReview.net, 2023.
- [45] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J. Lou, W. Chen, Repocoder: repository-level code completion through iterative retrieval and generation, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 2471–2484.
- [46] Y. Ishibashi, Y. Nishimura, Self-organized agents: a LLM multi-agent framework toward ultra large-scale code generation and optimization, *CoRR*, arXiv:2404.02183 [abs], 2024.
- [47] J. White, S. Hays, Q. Fu, J. Spencer-Smith, D.C. Schmidt, Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design, in: *Generative AI for Effective Software Development*, Springer, 2024, pp. 71–108.
- [48] Y. Li, Z. Lin, S. Zhang, Q. Fu, B. Chen, J. Lou, W. Chen, Making language models better reasoners with step-aware verifier, in: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 5315–5333.
- [49] N. Petrovic, F. Pan, K. Lebiada, V. Zolfaghari, S. Kirchner, N. Purschke, M.A. Khan, V. Vorobev, A. Knoll, Synergy of large language model and model driven engineering for automated development of centralized vehicular systems, *CoRR*, arXiv:2404.05508 [abs], 2024.
- [50] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al., Graph of thoughts: solving elaborate problems with large language models, in: *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024*, AAAI Press, 2024, pp. 17682–17690.
- [51] B. Jin, C. Xie, J. Zhang, K.K. Roy, Y. Zhang, S. Wang, Y. Meng, J. Han, Graph chain-of-thought: augmenting large language models by reasoning on graphs, *CoRR*, arXiv:2404.07103 [abs], 2024.